

Understanding How a Neural Network Works Using R

In the simplest of terms, a neural network is a computer system modelled on the human nervous system. It is widely used in machine learning technology. R is an open source programming language, which is mostly used by statisticians and data miners. It greatly supports machine learning, for which it has many packages.



The neural network is the most widely used machine learning technology available today. Its algorithm mimics the functioning of the human brain to train a computational network to identify the inherent patterns of the data under investigation. There are several variations of this computational network to process data, but the most common is the *feedforward-backpropagation* configuration. Many tools are available for its implementation, but most of them are expensive and proprietary. There are at least 30 different packages of open source neural network software available, and of them, R, with its rich neural network packages, is much ahead.

R provides this machine learning environment under a strong programming platform, which not only provides the supporting computation paradigm but also offers enormous flexibility on related data processing. The open source version of R and the supporting neural network packages are very easy to install and also comparatively simple to learn. In this article, I will demonstrate machine learning using a neural network to solve quadratic equation problems. I have

chosen a simple problem as an example, to help you learn machine learning concepts and understand the training procedure of a neural network. Machine learning is widely used in many areas, ranging from the diagnosis of diseases to weather forecasting. You can also experiment with any novel example, which you feel can be interesting to solve using a neural network.

Quadratic equations

The example I have chosen shows how to train a neural network model to solve a set of quadratic equations. The general form of a quadratic equation is: $ax^2 + bx + c = 0$. At the outset, let us consider three sets of coefficients a , b and c and calculate the corresponding roots $r1$ and $r2$.

The coefficients are processed to eliminate the linear equations with negative values of the discriminant, i.e., when $b^2 - 4ac < 0$. A neural network is then trained with these data sets. Coefficients and roots are numerical vectors, and they have been converted to the data frame for further operations.